

Square Segmentation on BeagleBone AI-64 with TIDL 8.2 Setup Tutorial, Model Report, and Run Guide

Project documentation

June 9, 2026

Contents

1	Flash the Correct Image to eMMC	2
2	Board Runtime Check	2
3	Board and Host Version Match	3
3.1	Host Setup and Reproduction Flow	3
4	Dataset	4
5	Final Model	4
6	Export ONNX with Opset 11	5
7	YAML Workflow	5
8	Host-Side Commands	6
9	TIDL Compilation	6
10	Final Deploy Package	7
11	Board-Side Commands	8
12	Measured CPU versus TIDL Results	8
13	Qualitative Result	9
14	Recommended Usage	10

Abstract

This document shows the BeagleBone AI-64 TIDL setup tutorial and the final square-segmentation model report. It covers flashing the correct image, validating the TIDL runtime, designing a TIDL 8.2-compatible ONNX graph, compiling with 08_02_00_01-rc1, deploying to the board, running inference through YAML files, and comparing CPU versus TIDL accuracy and latency. The selected model is a fully convolutional ONNX graph using only `Conv` and `Relu`.

The source repository is available at github.com/maria0419/beaglebone-ai-64-model-exemple. The repository keeps the BeagleBone/TIDL 8.2 compiler flow generic and places this square segmentation model under `models/square_seg_32x13/` as one example model.

1 Flash the Correct Image to eMMC

Use the official BeagleBoard image: [bbai64-emmc-flasher-debian-11.7-xfce-edgeai-arm64-2023-08-05-10gb.img.xz](https://beagleboard.org/latest-images/bb-ai-64-emmc-flasher-debian-11.7-xfce-edgeai-arm64-2023-08-05-10gb.img.xz).

This image was selected because it is the most up-to-date image that already includes the BeagleBone AI-64 EdgeAI/TIDL runtime compatible with the TIDL 8.2 workflow. The eMMC flashing procedure was as follows:

1. Write the image to a microSD card.
2. Insert the microSD card into the BeagleBone AI-64.
3. Hold the boot button.
4. Power the board while holding the boot button during the start of boot.
5. Leave the board flashing the eMMC for about 15 minutes.
6. Power off the board, remove the microSD card, and boot again from eMMC.

After boot, the board responded over SSH at:

```
192.168.7.2
192.168.6.2
```

The default user for this image is:

```
user: debian
password: temppwd
```

2 Board Runtime Check

Confirm ONNX Runtime and the TIDL providers on the board:

```
python3 -c "import onnxruntime as ort; print(ort.__version__); print(ort.get_available_providers())"
```

Expected for the image used in this project:

```
1.7.0
['TIDLExecutionProvider', 'TIDLCompilationProvider', 'CPUExecutionProvider']
```

Confirm that the TIDL provider library exists:

```
find /usr -name 'libtidl_onnxrt_EP.so' 2>/dev/null
```

Expected:

```
/usr/lib/libtidl_onnxrt_EP.so
```

3 Board and Host Version Match

The TIDL compilation flow starts from the board runtime, not from the host. On the image selected in the flashing step, ONNX Runtime reported version 1.7.0 and these providers:

```
TIDLExecutionProvider
TIDLCompilationProvider
CPUExecutionProvider
```

Because the board runtime is fixed by that image, the host must compile the model with the matching TIDL 8.2 tools. The host-side stack used for this board image was:

```
edgeai-tidl-tools / tidl_tools 08_02_00_01-rc1
onnxruntime_tidl 1.7.0
Python 3.6 inside the compile container
```

3.1 Host Setup and Reproduction Flow

On a fresh Debian or Ubuntu host, install Docker first:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

After adding the user to the `docker` group, log out and log in again so the new group membership is applied. Then verify Docker without `sudo`:

```
docker info
```

Download and extract the TIDL 8.2 host tools used by this project:

```
mkdir -p downloads tools/tidl82
wget -O downloads/tidl_tools_08_02_00_01_rc1.tar.gz \
  https://github.com/TexasInstruments/edgeai-tidl-tools/releases/download/08_02_00_01-rc1/
  tidl_tools.tar.gz
tar -xzf downloads/tidl_tools_08_02_00_01_rc1.tar.gz -C tools/tidl82
```

Verify that the extracted tools contain the ONNX Runtime TIDL provider libraries:

```
test -f tools/tidl82/tidl_tools/libtidl_onnxrt_EP.so
test -f tools/tidl82/tidl_tools/tidl_model_import_onnx.so
```

Build the TIDL 8.2 compile container once:

```
docker build -t tidl82-onnx-compiler:08_02_00_01_rc1 docker/tidl82
```

The container is needed because the `onnxruntime_tidl` wheel for this release targets Python 3.6. The wrapper itself is model-agnostic; the YAML file selects the already-exported ONNX model, the calibration images, and the output directory.

The final square segmentation artifacts are produced in this order:

1. Train the example with `python3 src/train.py -config configs/train.yaml`. This writes `artifacts/model.pt` and `artifacts/metrics.json`. The model itself is defined inside the `model:` block in `configs/train.yaml`; changing `model.name` and `model.params` changes which model is trained.
2. Re-export and compile with `./scripts/tidl82_export_compile.sh configs/compile.yaml`. This wrapper first exports `artifacts/model.onnx` from `artifacts/model.pt` on the host, then compiles that ONNX and writes `artifacts/tidl/model-artifacts/square\_seg/`.

For another ONNX model, copy `configs/compile_tidl82_template.yaml`, edit the ONNX path, calibration image directory, input size, and output directory, then pass that YAML to the same wrapper. If `calibration_dir` is an absolute host path, the wrapper mounts it read-only into the container automatically. For the square segmentation model, the one-command wrapper eventually calls the compiler script with the compile preset inside the container:

```
python3 tools/compile_tidl.py --config models/square_seg_32x13/configs/compile.yaml
```

4 Dataset

The dataset contains grayscale TIFF image/mask pairs:

It was generated with the repository github.com/Maria0419/Cube-Detector.

Table 1: Dataset and tensor format.

Item	Value
Training set	20,000 pairs
Test set	5,000 pairs
Input image	128×128 , grayscale TIFF
Mask	binary, 0/255 TIFF
Input tensor	$1 \times 1 \times 128 \times 128$, NCHW, float32 in $[0, 1]$
Output tensor	logits, $1 \times 1 \times 128 \times 128$
Post-processing	sigmoid outside ONNX, then threshold

5 Final Model

The selected architecture was not the highest floating-point model overall on the GPU. It was the best small model among the candidates tested through TIDL, based on the mask quality preserved after INT8 quantization on the real board.

Table 2: Final deployed model configuration.

Parameter	Value
Architecture	fully convolutional CNN
Hidden channels	32
Convolutional layers	13 total layers
Kernel size	3×3 for hidden layers
Final layer	1×1 convolution to one logit channel
Activations	ReLU only
Allowed ONNX ops	Conv, Relu
Export opset	11

The ONNX export check reported only **Conv** and **Relu**, with maximum absolute difference between PyTorch and ONNX Runtime CPU of 1.4496×10^{-4} on the export check tensor.

The exported ONNX file is:

```
artifacts/model.onnx
```

The final TIDL artifact files are:

```
logits_tidl_net.bin
logits_tidl_io_1.bin
```

6 Export ONNX with Opset 11

The export uses a fixed-shape dummy tensor and opset 11. For PyTorch versions with the newer exporter, the legacy exporter can be forced when needed:

```
torch.onnx.export(
    model,
    dummy_input,
    "artifacts/model.onnx",
    export_params=True,
    opset_version=11,
    do_constant_folding=True,
    input_names=["input"],
    output_names=["logits"],
    dynamic_axes=None,
    dynamo=False,
    external_data=False,
)
```

In this example, graph validation is part of the same export step. Running `python3 src/export_onnx.py -config configs/export.yaml` exports the ONNX file, checks that the graph is valid, confirms that the operator set matches this project, runs the CPU ONNX Runtime smoke test, and writes the summary to `artifacts/onnx_report.json`.

7 YAML Workflow

The workflow was simplified so that long command-line argument lists are no longer needed. Table 3 lists the YAML files in this example and what each one is for.

Table 3: YAML files used in the example workflow.

YAML file	What it is for
<code>configs/train.yaml</code>	Main training preset. It defines dataset paths, shared training settings, and the <code>model:</code> block used by the example.
<code>configs/train_32x13.yaml</code>	Candidate-specific training preset used to reproduce the selected 32x13 run.
<code>configs/export.yaml</code>	ONNX export preset. It reads <code>artifacts/model.pt</code> and writes <code>artifacts/model.onnx</code> .
<code>configs/compile.yaml</code>	TIDL compile preset. It selects the exported ONNX, calibration images, and the directory where the compiled TIDL artifacts are written.
<code>configs/run_cpu_board.yaml</code>	CPU inference preset for the packaged example layout. The inference script reads the image path, output path, and threshold from this YAML.
<code>configs/run_tidl.yaml</code>	TIDL inference preset for the packaged example on the board. The inference script reads the image path, TIDL artifacts path, and threshold from this YAML.

8 Host-Side Commands

From the project directory, the host-side flow for reproducing the selected model is:

```
python3 src/train.py --config configs/train_32x13.yaml
./scripts/tidl82_export_compile.sh configs/compile.yaml
```

The important dependency between these steps is that the compile YAML does not define the model architecture or training recipe. It only points the compiler to an already-exported ONNX file and to a representative calibration image set.

Before running the compile step, the user must set the calibration image directory in `configs/compile.yaml`. Those images are the representative inputs used for TIDL INT8 calibration.

9 TIDL Compilation

The host-side TIDL compilation uses these effective settings:

Table 4: TIDL compilation settings.

Option	Value
Platform	J7
Version	8.2
Provider	<code>TIDLCompilationProvider</code>
Tensor bits	8
Accuracy level	1
Max subgraphs	16

Calibration frames are representative input images used only during TIDL INT8 calibration so the compiler can estimate activation ranges for quantization. They are not labels and they are not used for training.

For this model, the one-command wrapper eventually calls:

```
python3 tools/compile_tidl.py --config models/square_seg_32x13/configs/compile.yaml
```

The one-command wrapper first exports the ONNX model on the host and then launches the TIDL compile step. The compile stage still consumes an ONNX model that was already exported; it does not recreate the training run or choose the saved model state by itself.

The effective provider stack during compilation is:

```
providers = ['TIDLCompilationProvider', 'CPUExecutionProvider']
```

The important provider options are:

```
delegate_options = {
    'tidl_tools_path': '/opt/tidl_tools',
    'artifacts_folder': 'artifacts/tidl/model-artifacts/square_seg',
    'platform': 'J7',
    'version': '8.2',
    'tensor_bits': 8,
    'debug_level': 1,
    'max_num_subgraphs': 16,
    'deny_list': '',
    'accuracy_level': 1,
    'advanced_options:calibration_frames': 128,
    'advanced_options:calibration_iterations': 5,
    'advanced_options:add_data_convert_ops': 3,
    'advanced_options:quantization_scale_type': 0,
    'advanced_options:high_resolution_optimization': 0,
    'advanced_options:pre_batchnorm_fold': 1,
    'ti_internal_nc_flag': 1601,
}
```

The compiler reported:

```
Final number of subgraphs created are : 1, - Offloaded Nodes - 25, Total Nodes - 25
ALL MODEL CHECK PASSED
```

The compiler also emitted a MSMC warning from `ti_cnnperfsim.out`. In this project the warning did not block runtime execution on the real BeagleBone AI-64, but it should still be tracked as a deployment risk.

10 Final Deploy Package

The deploy package generated on the host is:

```
artifacts/square_seg_32x13_tidl82.tar.gz
```

Copy that tarball once from the host PC to the BeagleBone AI-64, then unpack it on the board before running the board-side commands:

```
scp artifacts/square_seg_32x13_tidl82.tar.gz debian@192.168.7.2:~
ssh debian@192.168.7.2
rm -rf square_seg_tidl_yaml
mkdir -p square_seg_tidl_yaml
tar -xzf square_seg_32x13_tidl82.tar.gz -C square_seg_tidl_yaml
cd square_seg_tidl_yaml
```

Its relevant content is:

```
model.onnx
run_tidl.py
config_utils.py
```

```
configs/run_cpu_board.yaml
configs/run_tidl.yaml
test_images/img_0001_image.tif
artifacts/tidl/model-artifacts/square_seg/*
```

The local TIDL artifact folder is:

```
artifacts/tidl/model-artifacts/square_seg_32x13
```

The archive extraction can emit timestamp warnings when the board clock is behind the host clock. Those warnings do not affect inference.

11 Board-Side Commands

After copying or extracting the package on the board, the short commands inside the deployed package are:

The packaged example includes `test_images/img_0001_image.tif` as a sample input, but in normal use the user should edit the `image:` field in `configs/run_cpu_board.yaml` or `configs/run_tidl.yaml` to point to the test image they actually want to run. The script itself is YAML-driven; for inference it only needs the selected `-config`.

```
python3 run_tidl.py --config configs/run_cpu_board.yaml
sudo python3 run_tidl.py --config configs/run_tidl.yaml
```

From the host PC, the generated masks can be copied back for inspection with:

```
mkdir -p models/square_seg_32x13/artifacts/board_results
scp debian@192.168.7.2:~/square_seg_tidl_yaml/square_seg_32x13_tidl82/artifacts/cpu_mask.
  png models/square_seg_32x13/artifacts/board_results/
scp debian@192.168.7.2:~/square_seg_tidl_yaml/square_seg_32x13_tidl82/artifacts/tidl_mask.
  png models/square_seg_32x13/artifacts/board_results/
```

These exact YAML-based commands were tested successfully on the BeagleBone AI-64. The TIDL run log contained:

```
libtidl_onnxrt_EP loaded
Final number of subgraphs created are : 1, - Offloaded Nodes - 25, Total Nodes - 25
```

12 Measured CPU versus TIDL Results

All board measurements in this section use `img_0001_image.tif`, the corresponding binary label, and the same exported ONNX model.

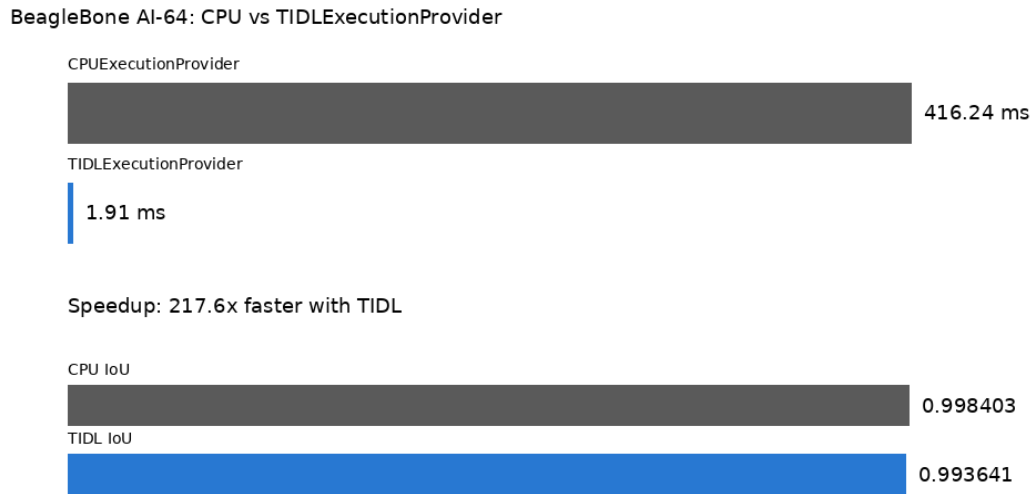
Table 5: Board accuracy and latency for the final 32x13 model.

Provider	Threshold	IoU	Dice	Mean time	Median time
CPUExecutionProvider	0.01	0.998403	0.999200	416.24 ms	416.33 ms
TIDLExecutionProvider	0.99	0.993641	0.996808	1.91 ms	1.98 ms

The measured speedup is approximately:

$$\frac{416.24 \text{ ms}}{1.91 \text{ ms}} \approx 217.6\times$$

The TIDL benchmark data for the measured run indicates roughly 1.43 ms of subgraph processing time between `ts:subgraph_logits_proc_start` and `ts:subgraph_logits_proc_end`. The remaining wall time comes from copies and runtime overhead.



Measured on `img_0001`, 128x128 grayscale segmentation. CPU: 50 runs. TIDL: 20 runs. Thresholds tuned per provider.

Figure 1: CPU versus TIDL latency and IoU on the measured BeagleBone AI-64 sample.

13 Qualitative Result

Figure 2 compares the input image, ground-truth mask, CPU mask, TIDL mask, and a TIDL error map. In the error map, green pixels are true positives, red pixels are false positives, and blue pixels are false negatives.

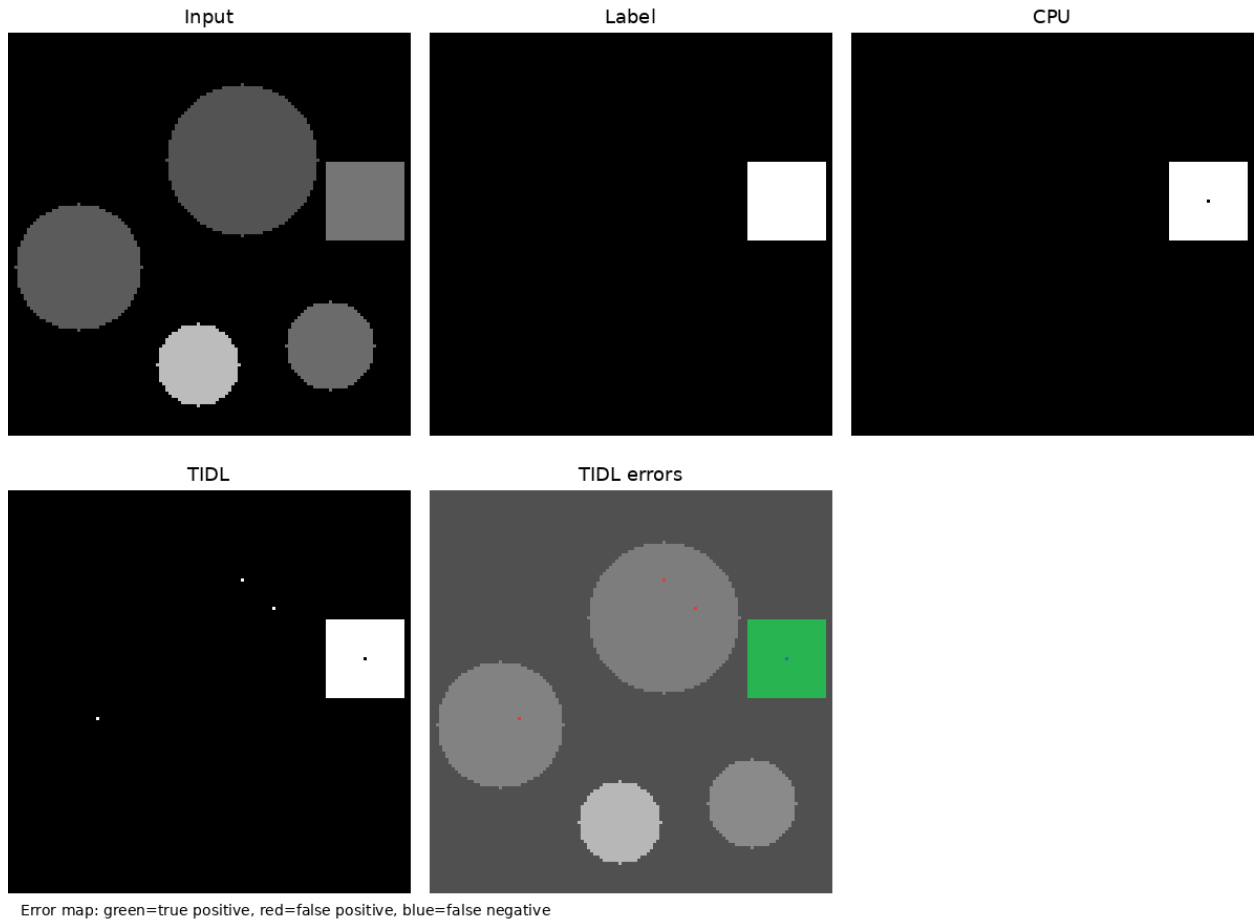


Figure 2: Qualitative comparison of CPU and TIDL outputs for the selected model.

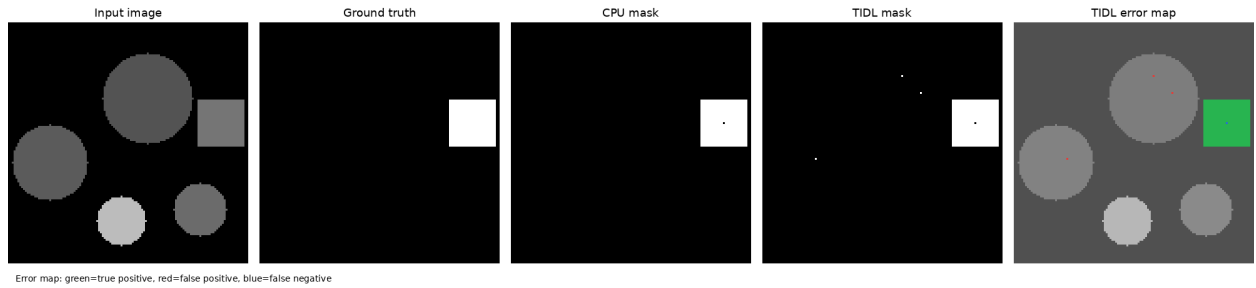


Figure 3: Wide qualitative view with the same CPU and TIDL masks.

14 Recommended Usage

To reproduce the deployed model from scratch:

1. On the host, train the selected candidate:

```
python3 src/train.py --config configs/train_32x13.yaml
```

2. On the host, export the trained model state to ONNX:

```
python3 src/export_onnx.py --config configs/export.yaml
```

3. On the host, compile the exported ONNX into TIDL artifacts:

```
./scripts/tidl82_onnx_compile.sh configs/compile.yaml
```

4. Copy `artifacts/square_seg_32x13_tidl82.tar.gz` from the host PC to the board and unpack it there.
5. On the board, run CPU inference from the unpacked example directory:

```
python3 run_tidl.py --config configs/run_cpu_board.yaml
```

6. On the board, run TIDL inference:

```
sudo python3 run_tidl.py --config configs/run_tidl.yaml
```

If you are reusing the already released model state and ONNX file, the training step can be skipped and the workflow starts at export or compile, depending on which artifacts you already have.